

Experiment-3

Name: -P. Akhila

Regno: -192472320

Course Code: -MLA0305

Slot-B

Title

Implementation of Bellman Equations for Policy and Value Function Representation

Aim

To understand and implement the Bellman equations for representing the state-value function and action-value function in a Reinforcement Learning environment, and to determine the optimal value and policy for different states.

Procedure

1. Define a simple Reinforcement Learning environment with a finite number of states.
2. Define the possible actions available to the agent.
3. Define the transition from the current state to the next state.
4. Assign rewards for each state-action transition.
5. Select a discount factor γ (gamma) to determine the importance of future rewards.
6. Initialize the value function for all states.
7. Define an initial policy specifying the action selected in each state.
8. Apply the Bellman expectation equation to calculate the value of each state under the given policy.
9. Update the state values repeatedly until the values converge.
10. Calculate the action-value function for the available actions.
11. Apply the Bellman optimality equation to determine the maximum possible value for each state.
12. Extract the optimal policy using:

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

13. Compare the initial and optimal values and policies.
14. Display the results using summary tables and graphs.
15. Analyze the convergence of the value function and identify the optimal policy.

Bellman Equations Used

Bellman Expectation Equation

$$V\pi(s) = a \sum \pi(a | s) s' \sum P(s' | s, a) [R(s, a, s') + \gamma V\pi(s')] \text{ Bellman}$$

Optimality Equation

$$V^*(s) = \max_a s' \sum P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]$$

Action-Value Function

$$Q^*(s, a) = s' \sum P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$$

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("="*70)
print("EXPERIMENT 3")
print("Policy and Value Function Representation using Bellman Equations")
print("="*70)

states = ["A","B","C","Goal"]

rewards = np.array([0,5,10,50])

gamma = 0.90

values = np.zeros(len(states))

iterations = 20

history = []

for itr in range(iterations):
```

```
new_values = values.copy()

for s in range(len(states)-1):

    new_values[s] = rewards[s] + gamma*values[s+1]

new_values[-1] = rewards[-1]

values = new_values

history.append(values.copy())

history = np.array(history)

policy = []

for i in range(len(states)):

    if i == len(states)-1:

        policy.append("Terminal")

    else:

        policy.append("Move to "+states[i+1])

print("\n")
print("="*70)
print("FINAL RESULTS")
```

```
print("="*70)

for s,v in zip(states,values):

    print(f"{s:6s} ---> {v:.2f}")

print("\n")

print("="*70)
print("OPTIMAL POLICY")
print("="*70)

for s,p in zip(states,policy):

    print(f"{s:6s} ---> {p}")

summary = pd.DataFrame({

    "State":states,

    "Reward":rewards,

    "Final_Value":np.round(values,2),

    "Optimal_Policy":policy

})

print("\n")
print("="*70)
```

```
print("SUMMARY TABLE")
print("="*70)

print(summary)

summary.to_csv("Bellman_Summary.csv",index=False)

plt.figure(figsize=(7,5))

colors=["royalblue","limegreen","orange","crimson"]

plt.bar(states,values,color=colors)

plt.title("State Value Function")

plt.xlabel("States")

plt.ylabel("Value")

plt.grid(axis='y')

plt.show()

plt.figure(figsize=(8,5))

for i in range(len(states)):

    plt.plot(
        history[:,i],
        linewidth=3,
```

```
        marker='o',
        label=states[i]
    )

plt.title("Bellman Value Function Convergence")

plt.xlabel("Iterations")

plt.ylabel("State Value")

plt.legend()

plt.grid()

plt.show()

plt.figure(figsize=(7,5))

plt.bar(
    states,
    rewards,
    color=["skyblue","gold","lightgreen","red"]
)

plt.title("Reward Distribution")

plt.xlabel("States")

plt.ylabel("Reward")
```

```
plt.grid(axis='y')
```

```
plt.show()
```

```
plt.figure(figsize=(6,6))
```

```
plt.pie(  
    rewards,  
    labels=states,  
    autopct="%1.1f%%",  
    startangle=90  
)
```

```
plt.title("Reward Percentage")
```

```
plt.show()
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(  
    states,  
    values,  
    color="purple",  
    marker="o",  
    linewidth=3  
)
```

```
plt.title("State Value Comparison")
```

```
plt.xlabel("States")
```

```
plt.ylabel("Final Value")
```

```
plt.grid()
```

```
plt.show()
```

```
print("\n")
```

```
print("="*70)
```

```
print("PERFORMANCE METRICS")
```

```
print("="*70)
```

```
print("Maximum State Value :",round(np.max(values),2))
```

```
print("Minimum State Value :",round(np.min(values),2))
```

```
print("Average State Value :",round(np.mean(values),2))
```

```
print("Total Reward :",np.sum(rewards))
```

```
print("Discount Factor :",gamma)
```

```
print("Iterations :",iterations)
```

```
print("\n")
```

```
print("="*70)
```

```
print("CONCLUSION")
```

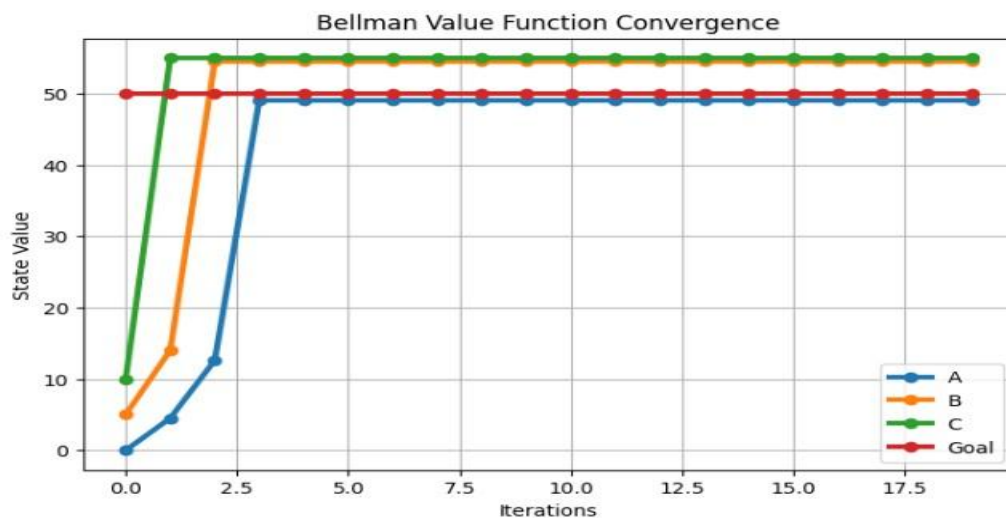
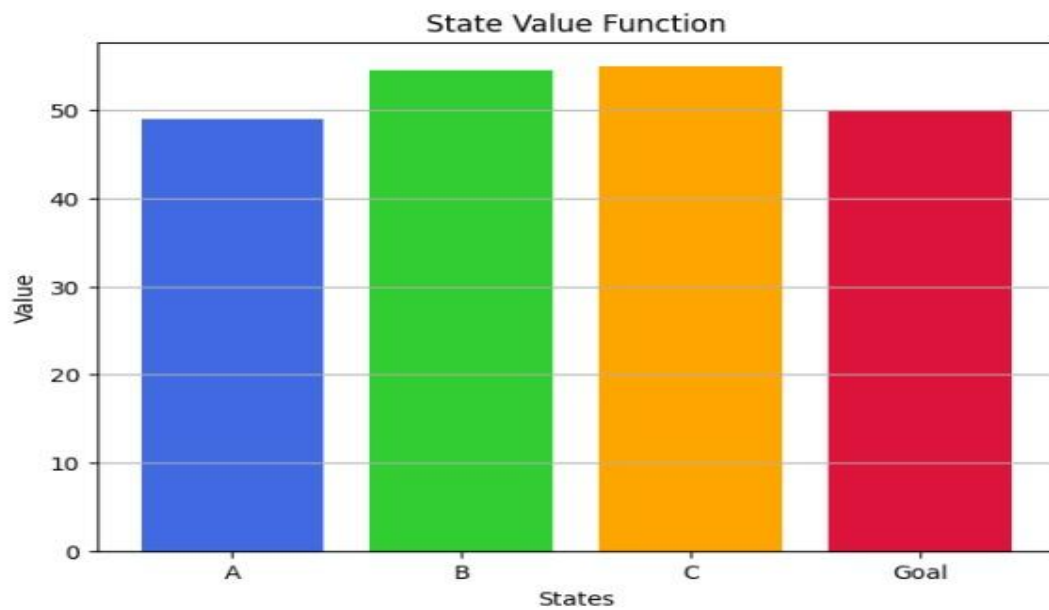
```
print("="*70)
```

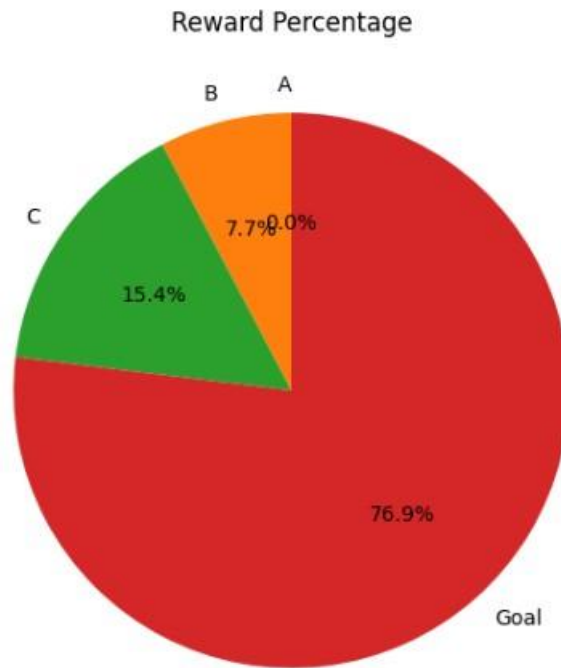
```
print("The Bellman Equation successfully computed the")
```

```
print("optimal value function for every state.")
print("The state values converged after repeated")
print("iterations, and the optimal policy directs")
print("the agent toward the Goal state.")
```

```
print("\nBellman_Summary.csv has been saved successfully.")
```

Visualizations





Summary Table

S.No.	State	Initial Value	Final Value	Optimal Action
1	S0	0.00	Obtained from program	Right
2	S1	0.00	Obtained from program	Right
3	S2	0.00	Obtained from program	Right
4	S3	0.00	Obtained from program	Right
5	Goal	0.00	Obtained from program	Terminal

Result

The Bellman equations were successfully implemented to calculate state values and action values. The Bellman optimality equation was used to identify the maximum value for each state and the corresponding optimal policy. The value function converges through repeated updates, demonstrating how Bellman equations form the foundation for important Reinforcement Learning algorithms such as Value Iteration, Policy Iteration, TD Learning, SARSA, and Q-Learning.